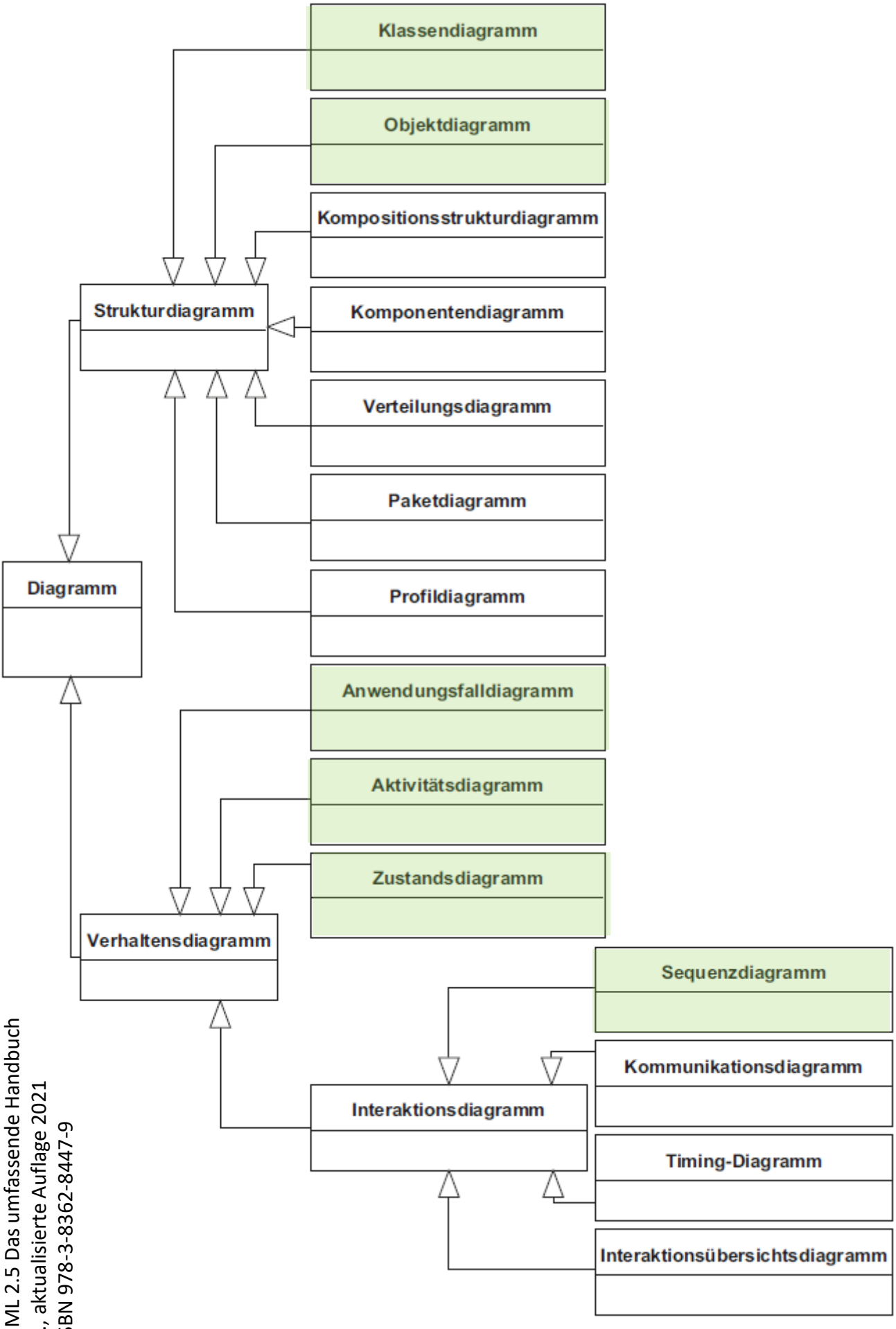
Übersicht über UML-Diagrammtypen



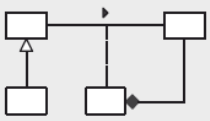
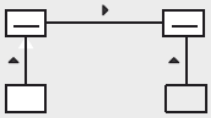
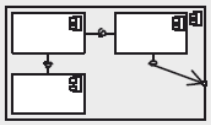
UML 2.5 Das umfassende Handbuch
7., aktualisierte Auflage 2021
ISBN 978-3-8362-8447-9

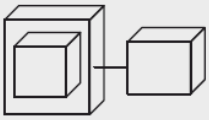
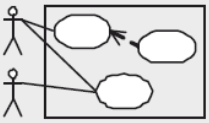
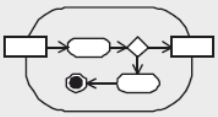
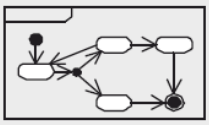
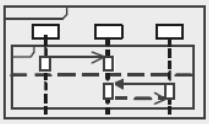
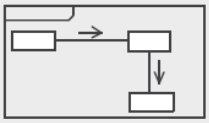
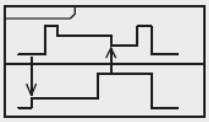
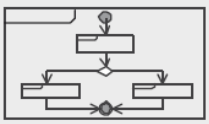
Übersicht über UML-Diagrammtypen

Quelle: UML2 glasklar, Praxiswissen für die UML-Modellierung, 4. Auflage 2012, ISBN 978-3-446-43057-0

Zuordnung der Diagrammtypen zu Struktur- und Verhaltensdiagrammen:

Strukturdiagramme	Verhaltensdiagramme	
		Interaktionsdiagramme
Klassendiagramm	Use-Case-Diagramm	Sequenzdiagramm
Paketdiagramm	Aktivitätsdiagramm	Kommunikationsdiagramm
Objektdiagramm	Zustandsautomat	Timingdiagramm
Kompositionsstrukturdiagramm		Interaktionsübersichtsdiagramm
Komponentendiagramm		
Verteilungsdiagramm		
Profildiagramm		

Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Klassendiagramm 	Aus welchen Klassen besteht mein System und wie stehen diese untereinander in Beziehung?	Beschreibt die statische Struktur des zu entwerfenden oder abzubildenden Systems. Enthält alle relevanten Strukturzusammenhänge und Datentypen. Bildet die Brücke zu den dynamischen Diagrammen.
Paketdiagramm 	Wie kann ich mein Modell so schneiden, dass ich den Überblick bewahre?	Organisiert das Systemmodell in größere Einheiten durch logische Zusammenfassung von Modellelementen. Modellierung von Abhängigkeiten.
Objektdiagramm 	Welche innere Struktur besitzt mein System zu einem bestimmten Zeitpunkt zur Laufzeit (Klassendiagrammschnappschuss)	Zeigt Objekte und deren Attributbelegungen zu einem bestimmten Zeitpunkt. Wird nur beispielhaft zur Veranschaulichung verwendet. Detailniveau wie im Klassendiagramm. Sehr gute Darstellung von Mengenverhältnissen.
Kompositionsstrukturdiagramm 	Wie sieht das Innenleben einer Klasse, einer Komponente, eines Systemteils aus?	Ideal für die Top-Down-Modellierung des Systems. Mittleres Detailniveau, zeigt Teile eines „Gesamtelements“ und deren Mengenverhältnisse.
Komponentendiagramm 	Wie werden meine Klassen zu wiederverwendbaren, verwaltbaren Komponenten zusammengefasst und wie stehen diese miteinander in Beziehung?	Zeigt Organisation und Abhängigkeiten einzelner technischer Systemkomponenten. Modellierung angebotener und benötigter Schnittstellen möglich.

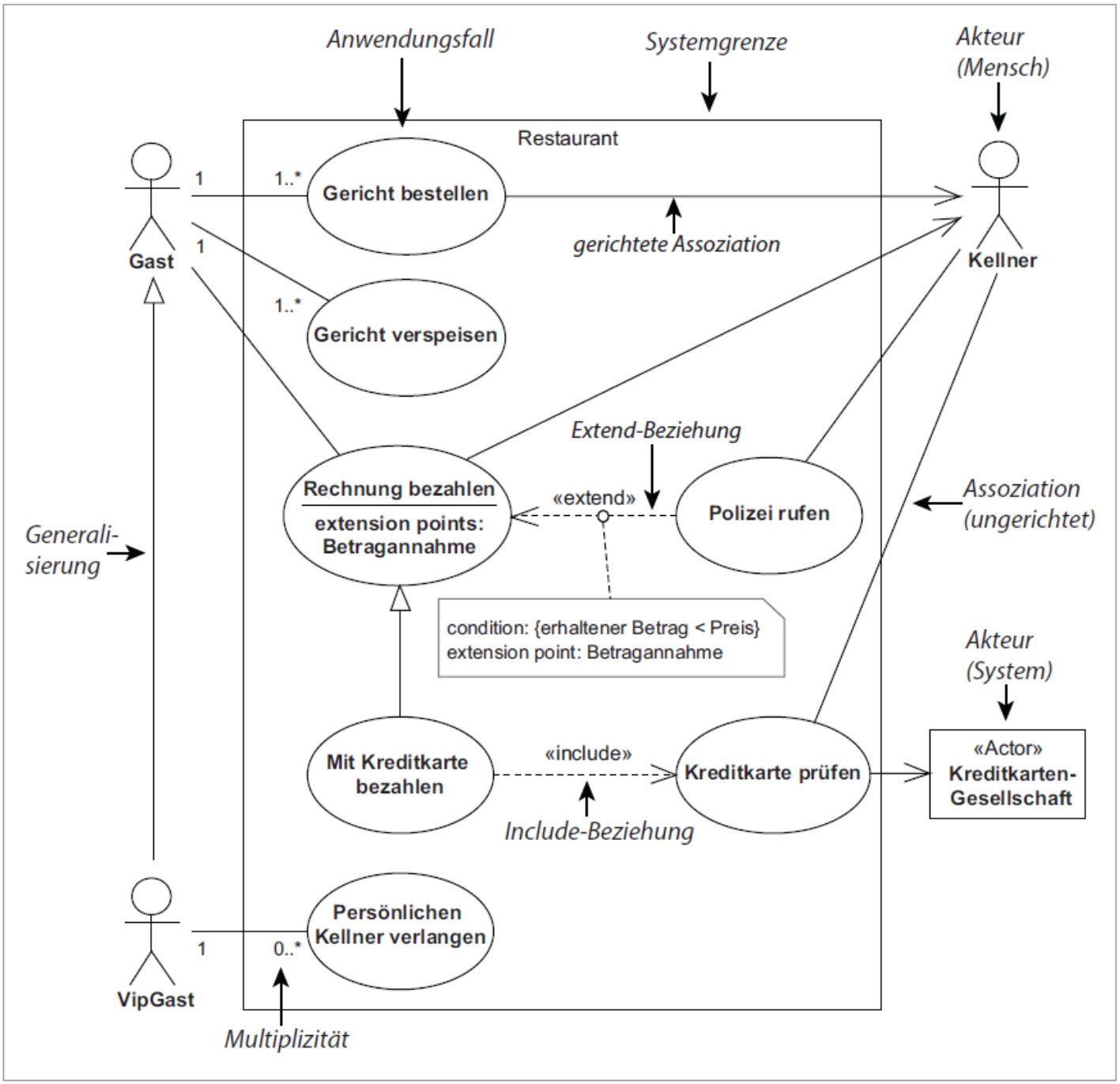
Diagrammtyp	Diese zentrale Frage beantwortet das Diagramm	Stärken
Verteilungsdiagramm 	Wie sieht das Einsatzumfeld (Hardware, Server, Datenbanken, ...) des Systems aus? Wie werden die Komponenten zur Laufzeit wohin verteilt?	Zeigt das Laufzeitumfeld des Systems mit den „greifbaren“ Systemteilen (meist Hardware). Hohes Abstraktionsniveau, kaum Notationselemente.
Use-Case-Diagramm 	Was leistet mein System für seine Umwelt (Nachbarsysteme, Stakeholder)?	Präsentiert die Außensicht auf das System. Geeignet zur Kontextabgrenzung. Hohes Abstraktionsniveau, einfache Notationsmittel.
Aktivitätsdiagramm 	Wie läuft ein bestimmter flussorientierter Prozess oder ein Algorithmus ab?	Sehr detaillierte Visualisierung von Abläufen mit Bedingungen, Schleifen, Verzweigungen. Parallelisierung und Synchronisation möglich.
Zustandsautomat 	Welche Zustände kann ein Objekt, eine Schnittstelle, ein Use Case, ... bei welchen Ereignissen annehmen?	Präzise Abbildung eines Zustandsmodells mit Zuständen, Ereignissen, Nebenläufigkeiten, Bedingungen, Ein- und Austrittsaktionen. Schachtelung möglich.
Sequenzdiagramm 	Wer tauscht mit wem welche Informationen in welcher Reihenfolge aus?	Stellt den zeitlichen Ablauf des Informationsaustausches zwischen Kommunikationspartnern dar. Schachtelung und Flusssteuerung (Bedingungen, Schleifen, Verzweigungen) möglich.
Kommunikationsdiagramm 	Wer kommuniziert mit wem? Wer „arbeitet“ im System zusammen?	Stellt den Informationsaustausch zwischen Kommunikationspartnern dar. Überblick steht im Vordergrund (Details und zeitliche Abfolge weniger wichtig).
Timingdiagramm 	Wann befinden sich verschiedene Interaktionspartner in welchem Zustand?	Visualisiert das exakte zeitliche Verhalten von Klassen, Schnittstellen, ... Geeignet für Detailbetrachtungen, bei denen es überaus wichtig ist, dass ein Ereignis zum richtigen Zeitpunkt eintritt.
Interaktionsübersichtsdiagramm 	Wann läuft welche Interaktion ab?	Verbindet Interaktionsdiagramme (Sequenz-, Kommunikations- und Timingdiagramme) auf Top-Level-Ebene. Hohes Abstraktionsniveau. Gut geeignet als Strukturierung der Interaktionsdiagramme.

Use-Case Diagramme

Anwendungsfalldiagramme

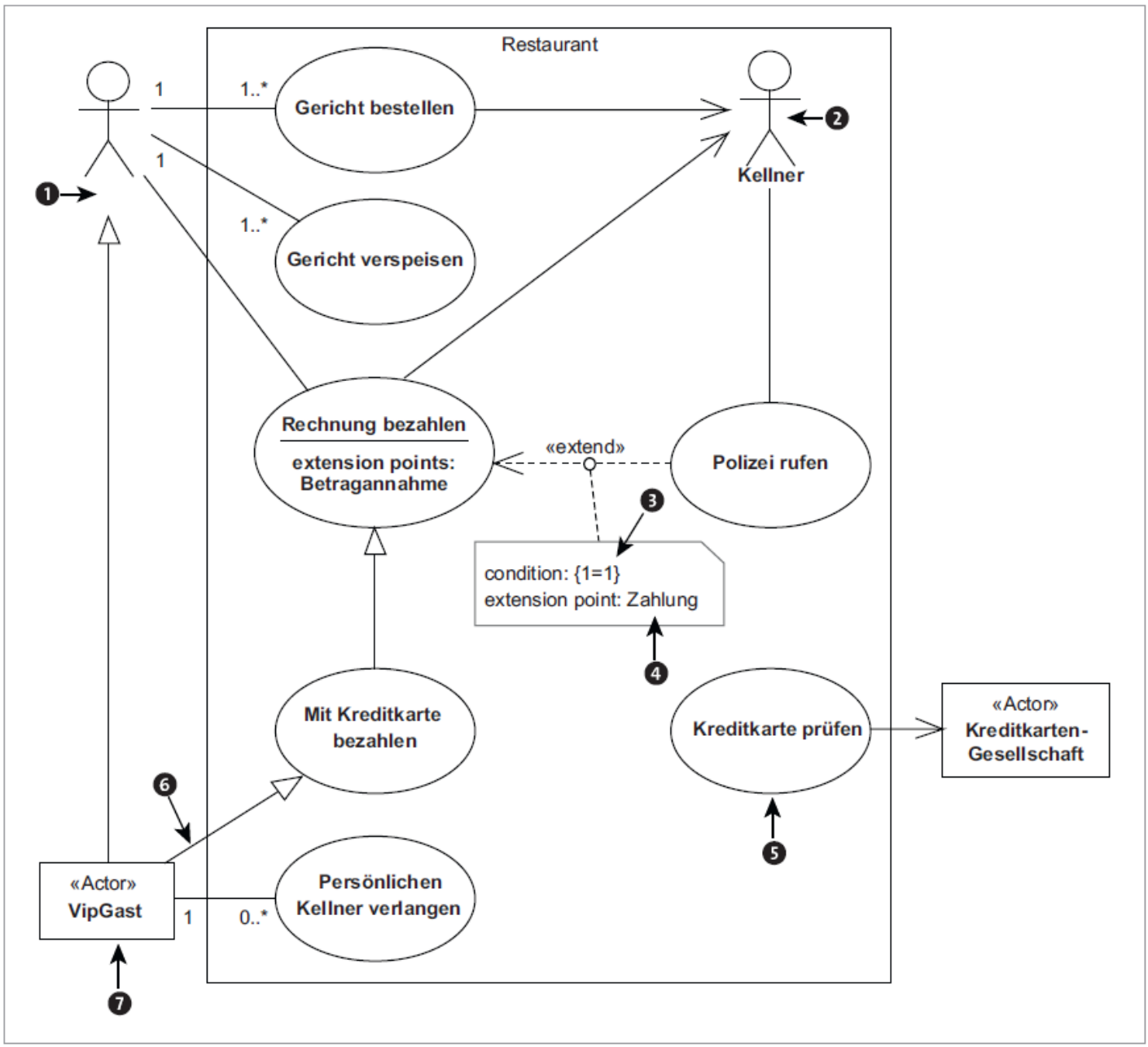
Aus dem UML-Handbuch:
Anwendungsfalldiagramme (engl. *Use Case Diagrams*) modellieren die Funktionalität des Systems auf einem hohen Abstraktionsniveau aus der sogenannten Black-Box-Sicht des Anwenders. Es werden nur die Anwendungsfälle definiert, die ein externer Anwender wahrnehmen kann und deren Ausführung ihm einen erkennbaren Nutzen bringt.
Die Modellierung beschreibt, *was* für Anwendungsfälle das System anbietet, und nicht, *wie* sie im System realisiert werden. So wird weder eine Reihenfolge modelliert, in der die Anwendungsfälle abgerufen werden sollten, noch was während der Anwendungsfälle im System eigentlich geschieht. Der Fokus wird darauf gelegt, was ein Anwender von dem System erwarten kann.

Use-Case Diagramm: Notationselemente



Tipp zur Eingabe von «include» und «extend», wenn nötig.
Sie verwenden französische Anführungszeichen, »guillements« genannt.
Eingabe mit ALT-Taste und Zahlensequenz auf dem Ziffernblock.
Alt+0171 -> **«** Alt+0187 -> **»**

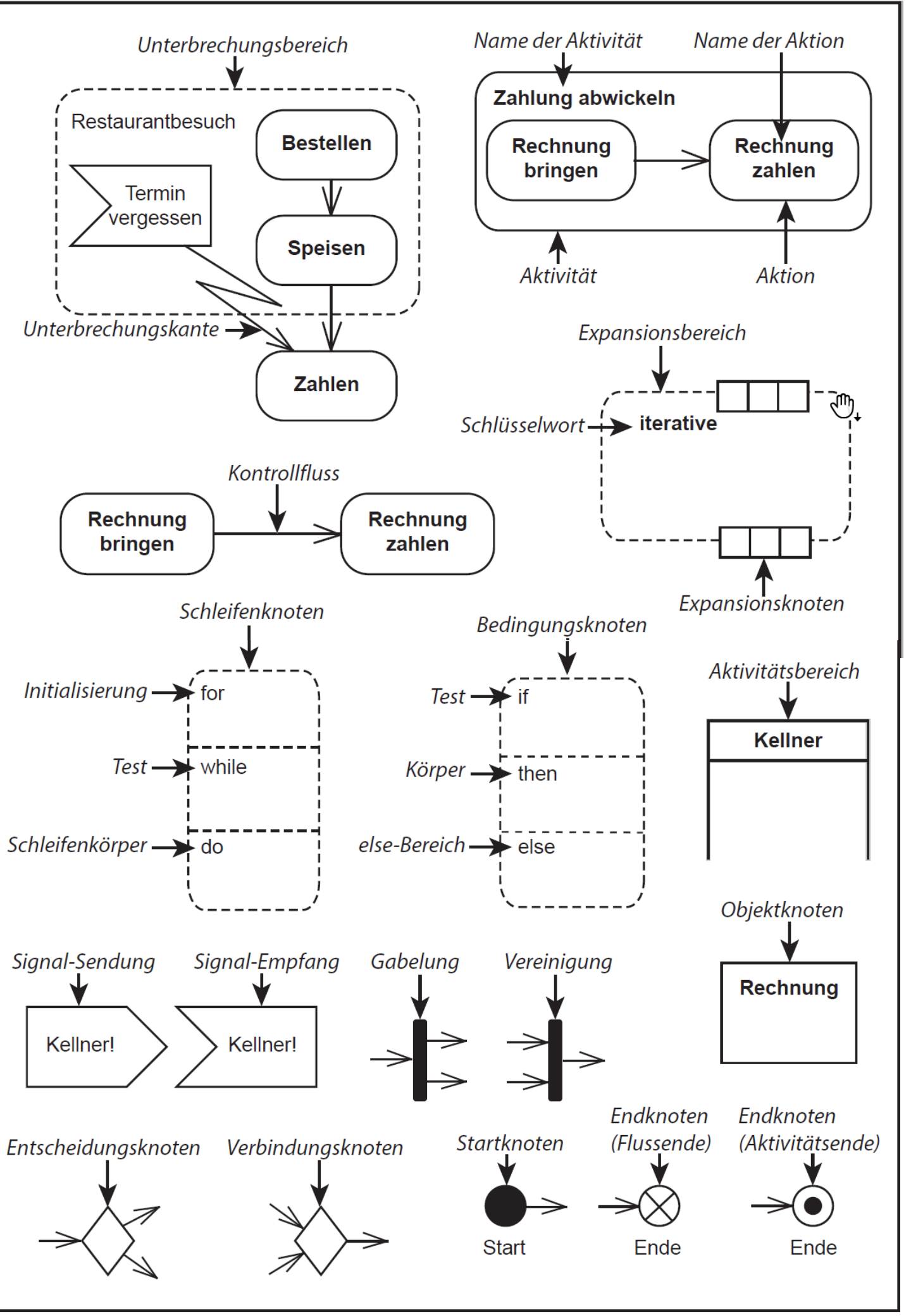
Use-Case Diagramm: Identifiziere die Fehler!



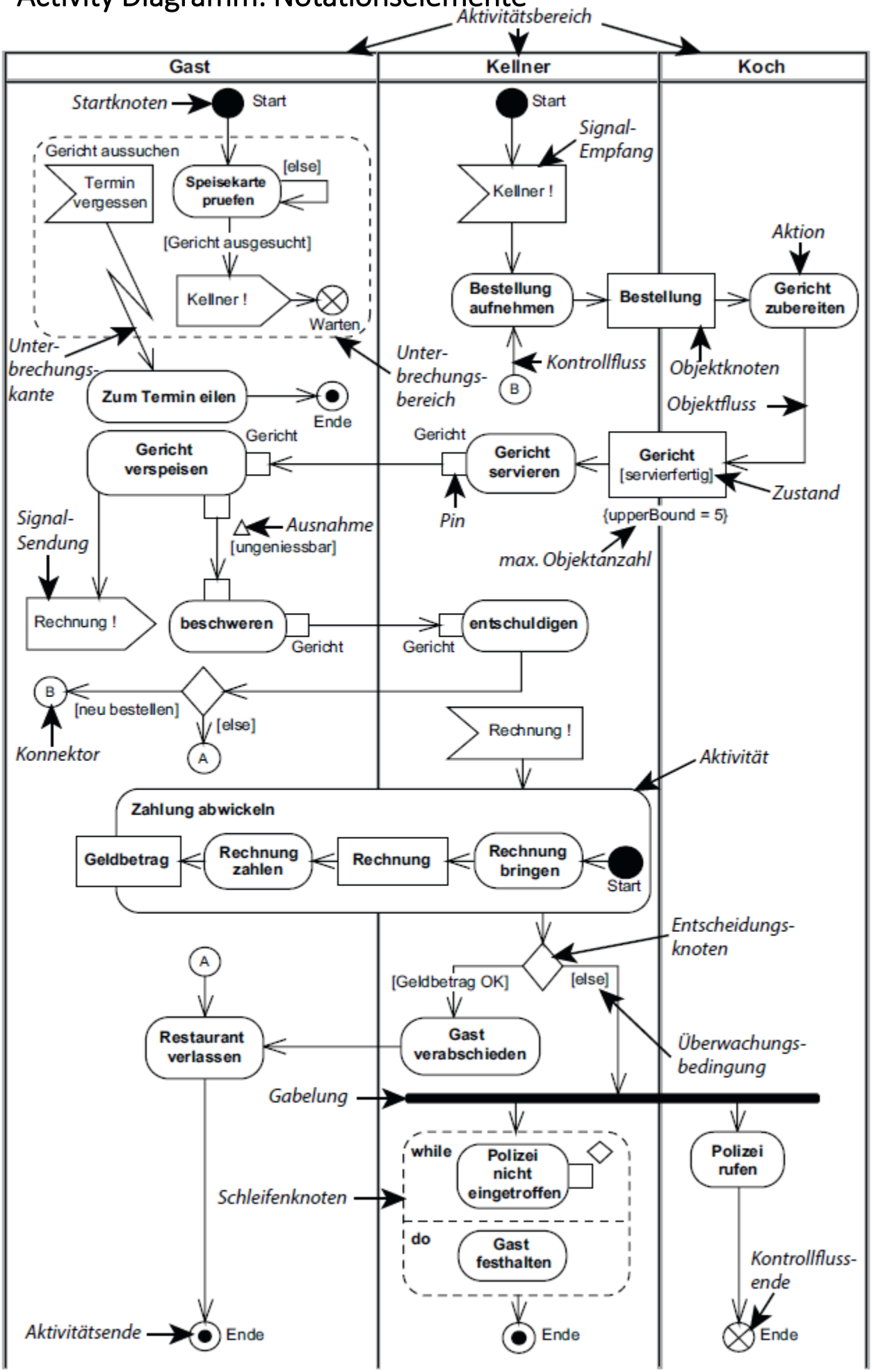
Activity Diagramme

Aktivitätsdiagramme

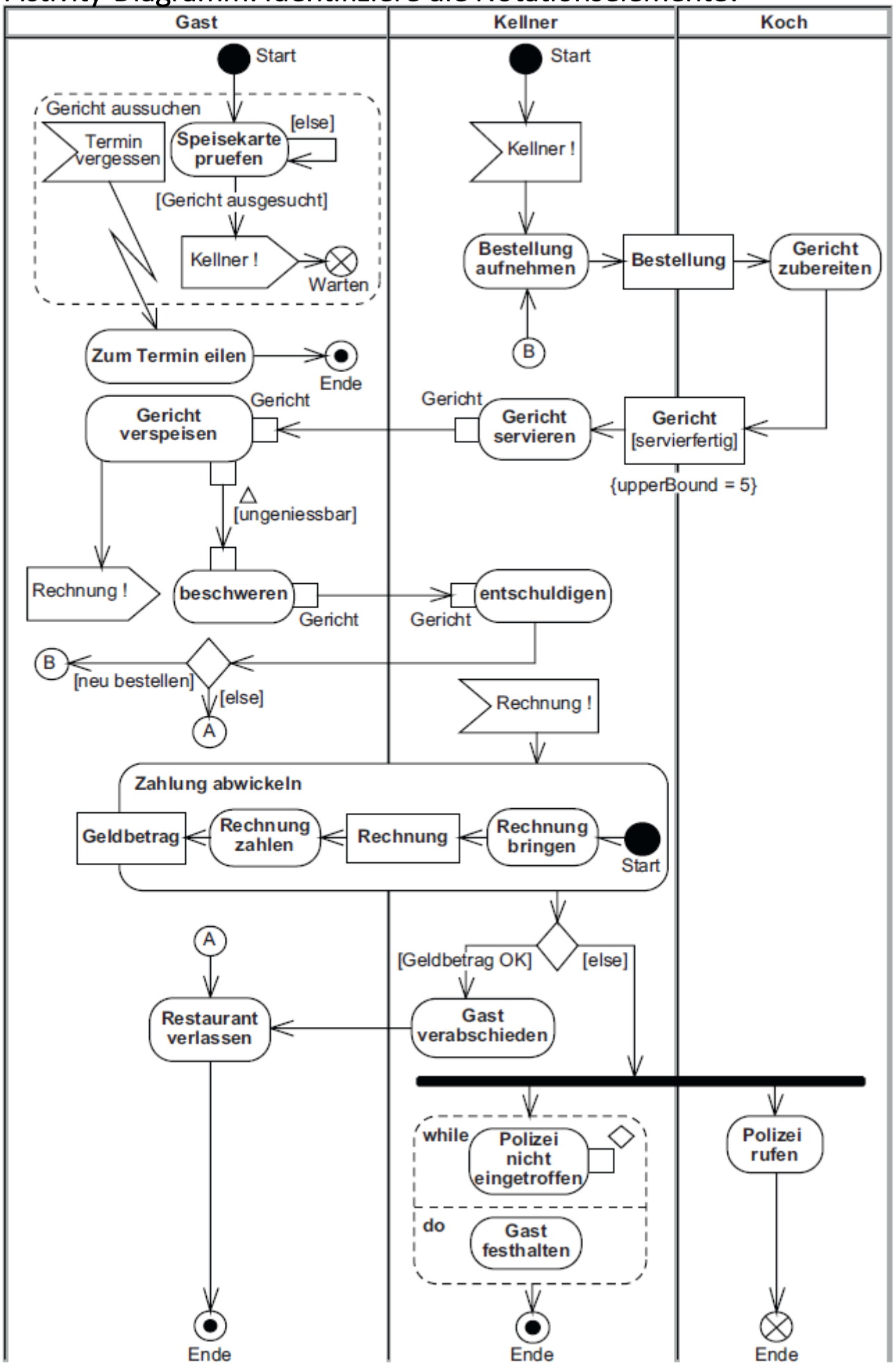
Aus dem UML-Handbuch:
Während der Analyse/Definition-Phase werden Aktivitätsdiagramme verwendet, um Geschäftsprozesse zu modellieren und zu analysieren. Sie zeigen die Reihenfolge der Prozesse und Tätigkeiten sowie mögliche Alternativabläufe auf.
In der Entwurfs-/Design-Phase bieten Aktivitätsdiagramme vielfältige Möglichkeiten zur Modellierung interner Systemprozesse. ... Aktivitätsdiagramme aus dieser Phase werden als Arbeitsvorlage für Programmierer entwickelt. Die Vielfalt und Mächtigkeit der Notationselemente kann daher voll ausgeschöpft werden, um das interne Verhalten der Software möglichst präzise zu beschreiben.


Unterbrechungsbereich
Restaurantbesuch
Bestellen
Termin vergessen
Speisen
Unterbrechungskante
Zahlen
Name der Aktivität
Name der Aktion
Zahlung abwickeln
Rechnung bringen
Rechnung zahlen
Aktivität
Aktion
Expansionsbereich
Schlüsselwort
iterative
Expansionsknoten
Kontrollfluss
Rechnung bringen
Rechnung zahlen
Schleifenknoten
Initialisierung
for
Test
while
Schleifenkörper
do
Bedingungsknoten
Test
if
Körper
then
else-Bereich
else
Aktivitätsbereich
Kellner
Objektknoten
Rechnung
Signal-Sendung
Signal-Empfang
Kellner!
Kellner!
Gabelung
Vereinigung
Entscheidungsknoten
Verbindungsknoten
Startknoten
Start
Endknoten (Flussende)
Ende
Endknoten (Aktivitätsende)
Ende

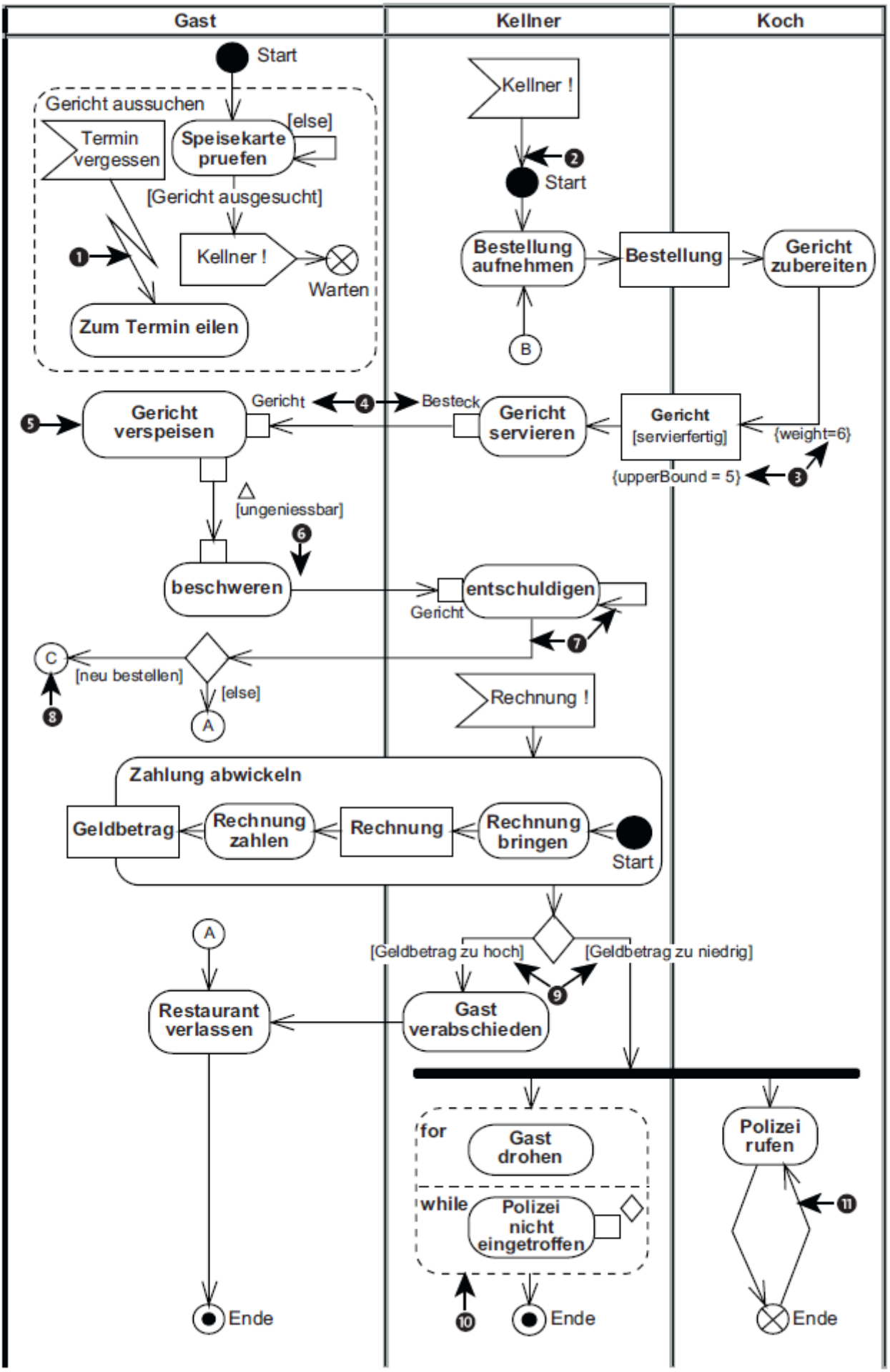
Activity Diagramm: Notationselemente



Activity-Diagramm: Identifiziere die Notationselemente!



Activity Diagramm: Was ist hier verkehrt?

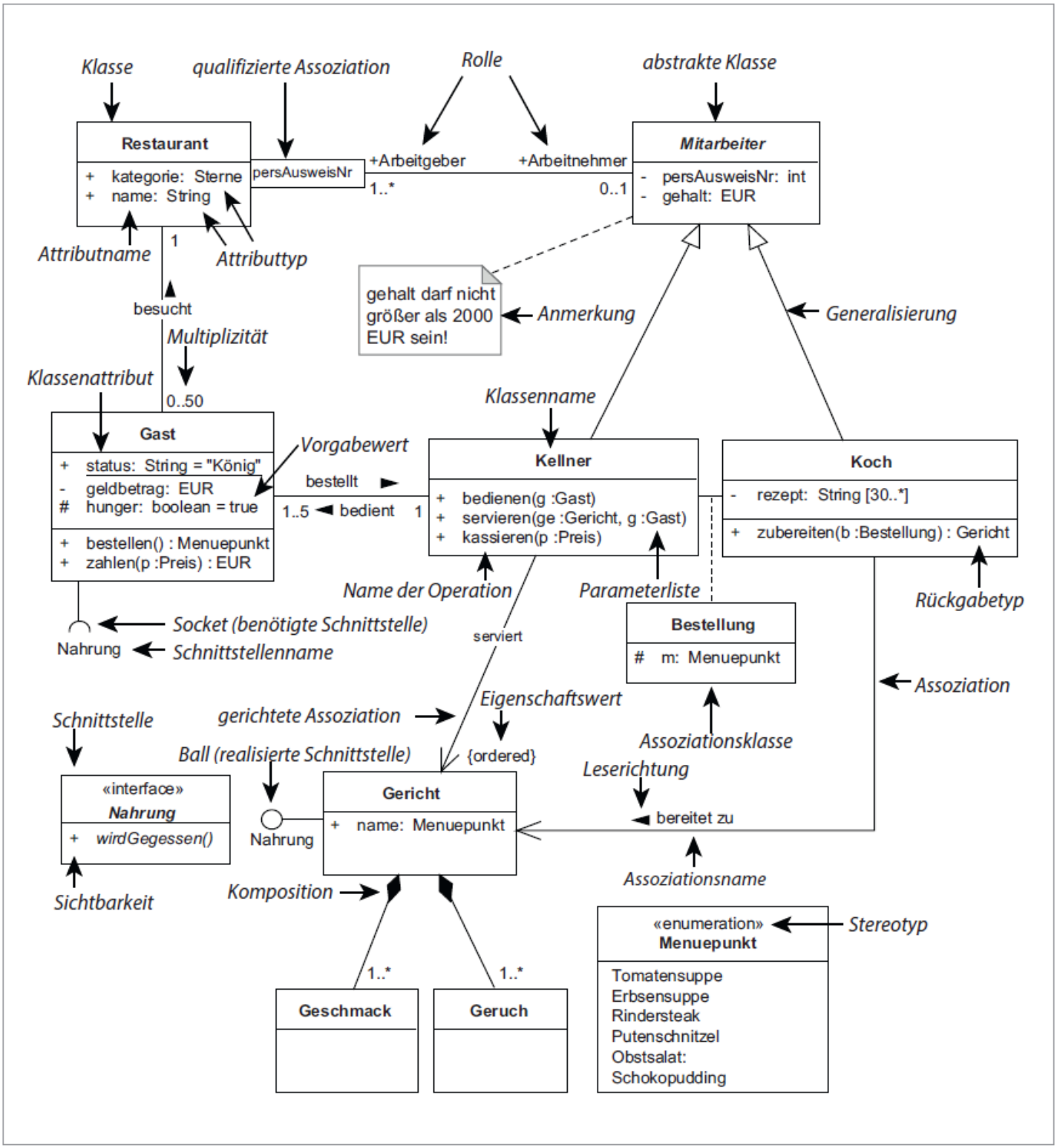


Class Diagramme

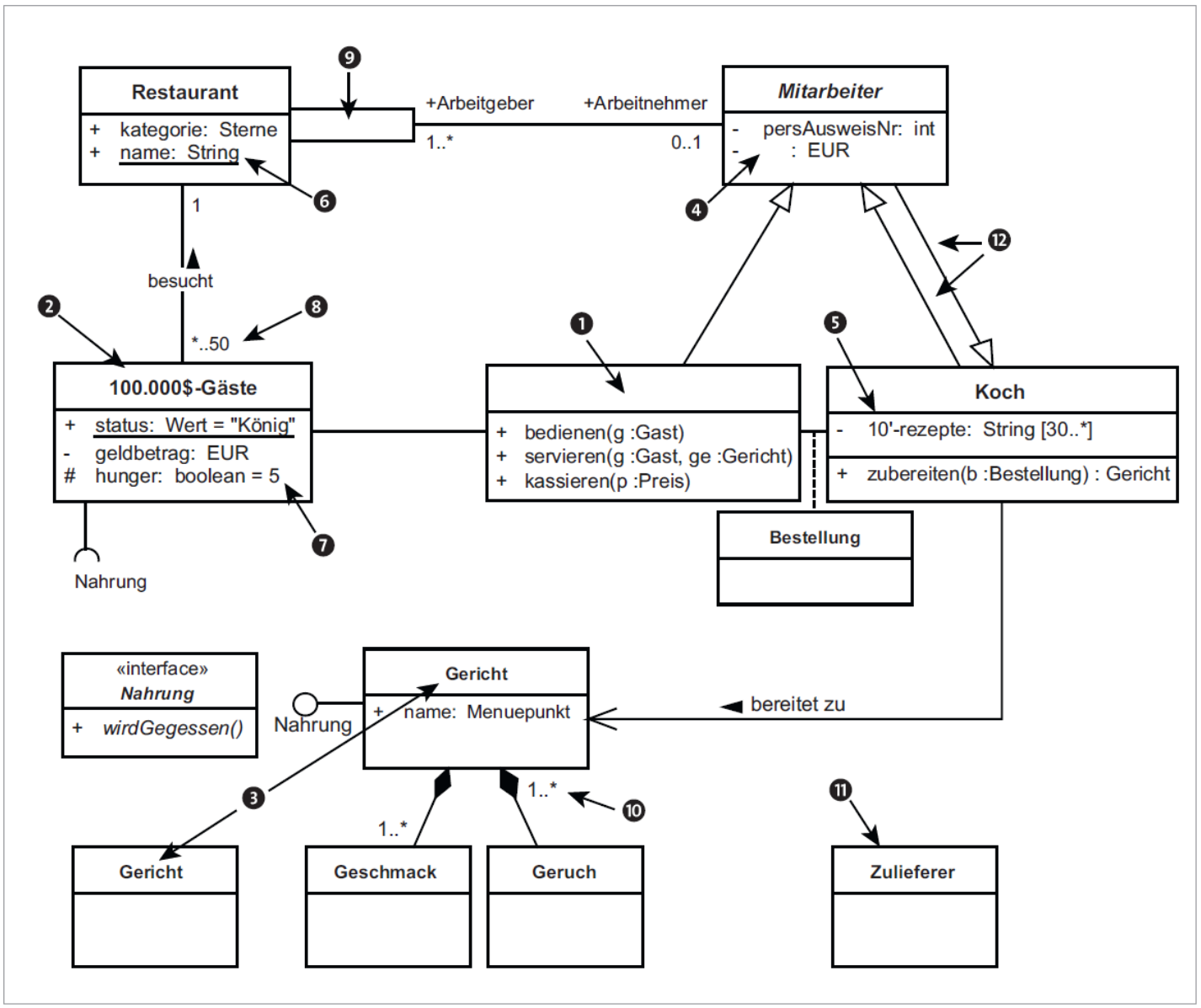
Klassendiagramme

Aus dem UML-Handbuch:
Klassendiagramme zeigen die **statischen Bestandteile und Attribute** von Systemen und welche **Beziehungen** sie untereinander einnehmen können. Bis auf die Definition von **Operationen** der Klassen werden jegliche dynamischen Aspekte außer Acht gelassen.
Klassendiagramme werden üblicherweise in den ersten beiden Phasen Analyse und Entwurf erstellt, können jedoch auch während der Phasen Implementierung, Test und Einsatz gewinnbringend verwendet werden.
…
Die in der Entwurfsphase modellierten Klassendiagramme sind im Vergleich zu denjenigen aus der Analyse-Phase viel detailreicher und schöpfen die Mächtigkeit der verfügbaren Notationselemente voll aus.

Class Diagramm: Notationselemente



Class Diagramm: Identifiziere die Fehler!



State Diagramme

Zustandsdiagramme

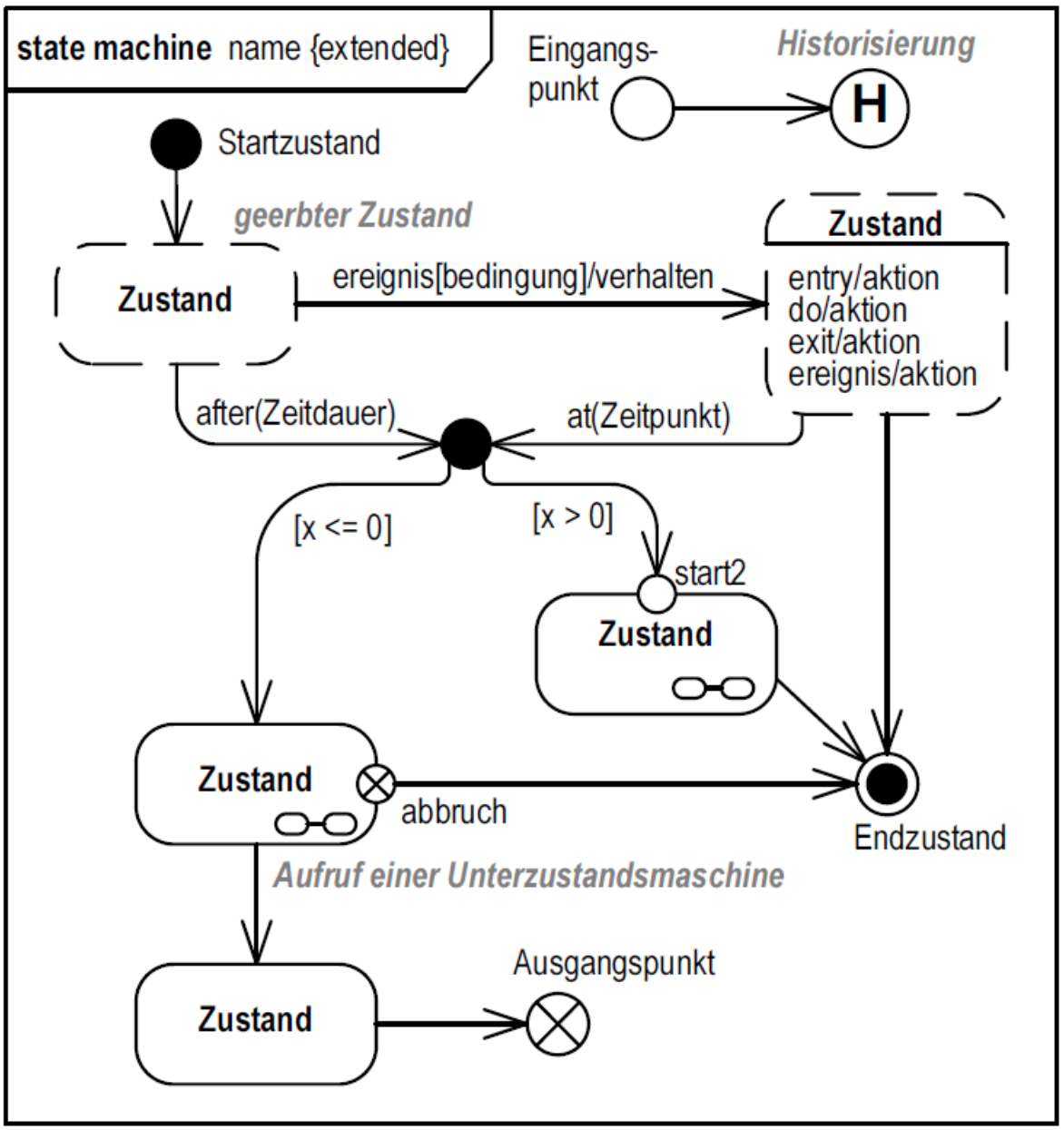
State Machine

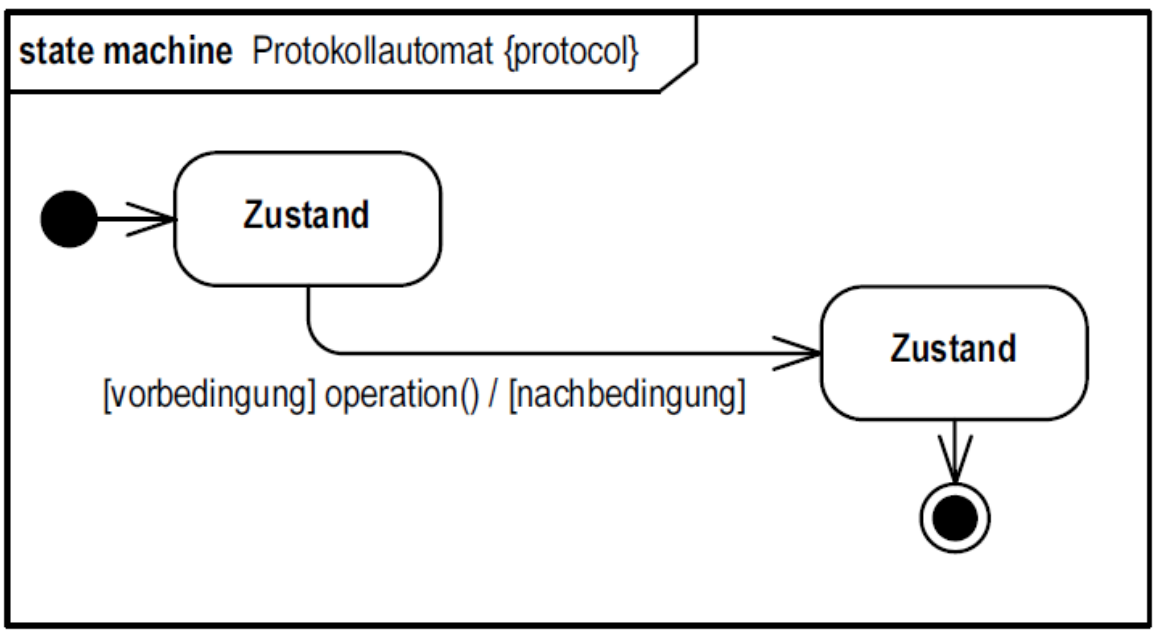
Sinngemäß aus dem UML-Handbuch:
Zustandsdiagramme konzentrieren sich auf die Reaktionen eines Systems.

Verhaltens-Zustandsdiagramme eignen sich sehr gut zur Modellierung des Verhaltens von Benutzeroberflächen, die üblicherweise nur auf Befehle von Benutzern reagieren und selbst keine eigenen Aktionen initiieren.

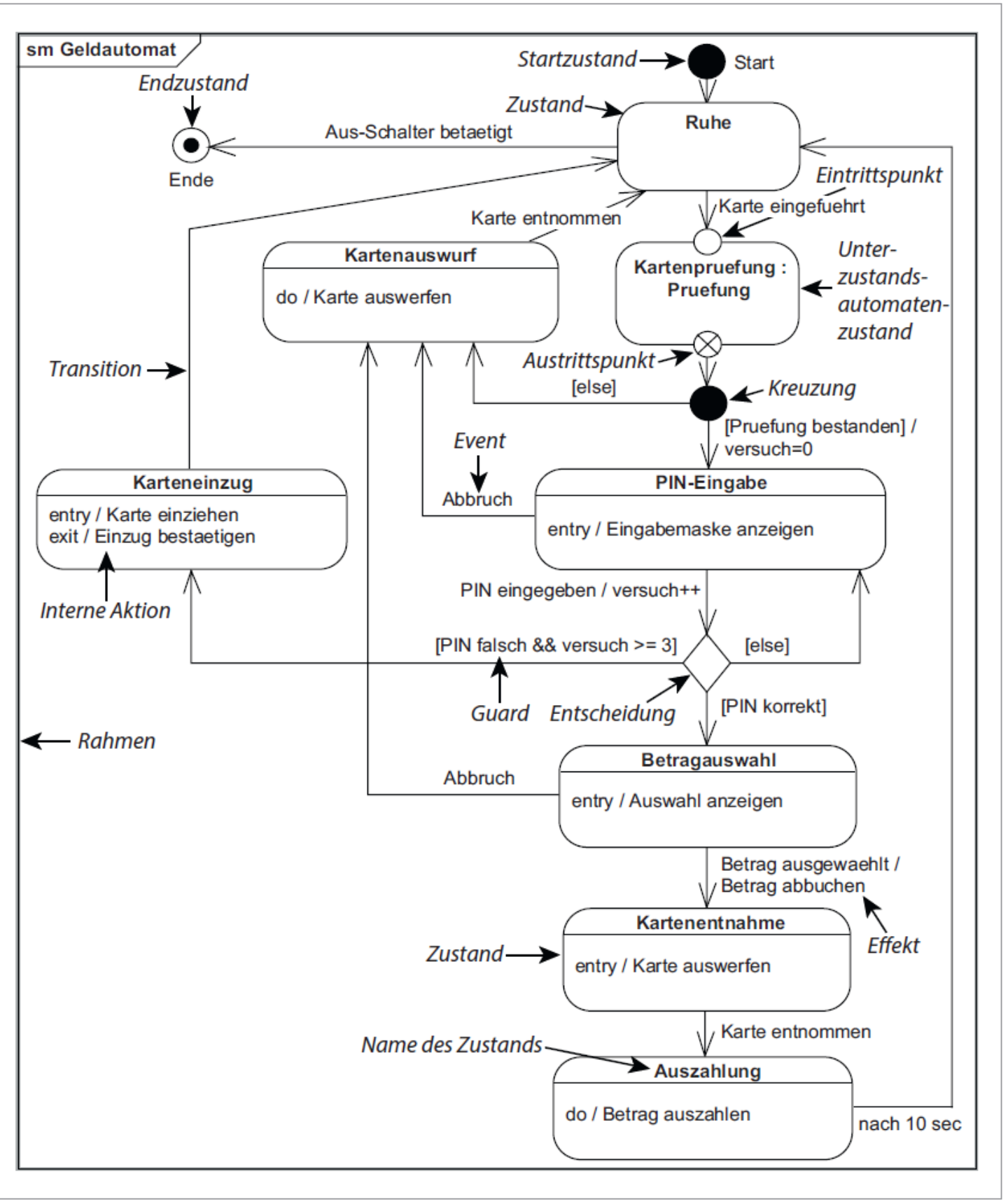
Protokoll-Zustandsdiagramme modellieren Kommunikationsprotokolle, die Vorgaben über die Reihenfolge und Voraussetzungen von Kommunikationsschritten definieren.

Verhaltenszustandsautomat vs. Protokollautomat

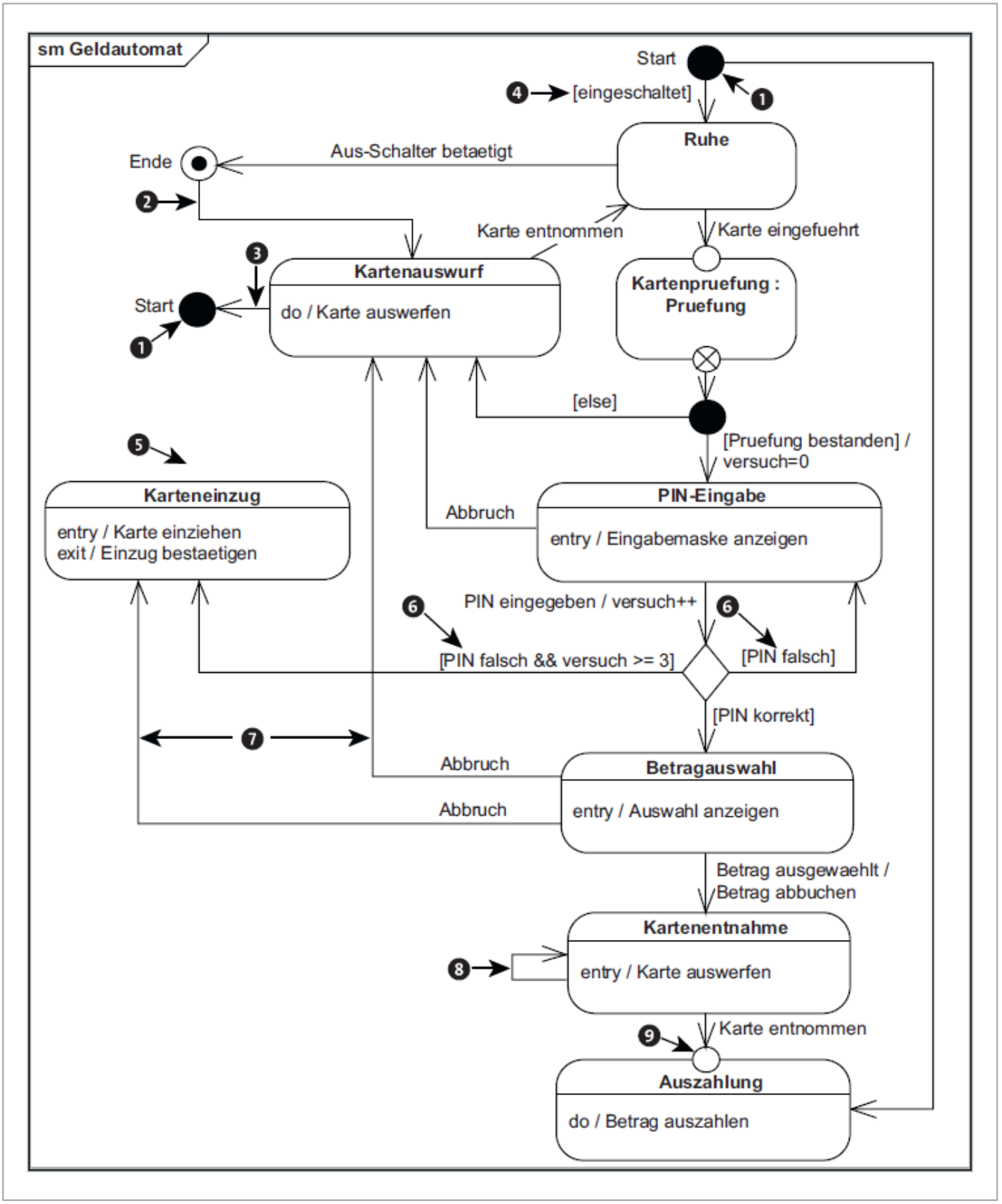




State Diagramm: Notationselemente



State Diagramm: Identifiziere die Fehler!



Sequence Diagramme

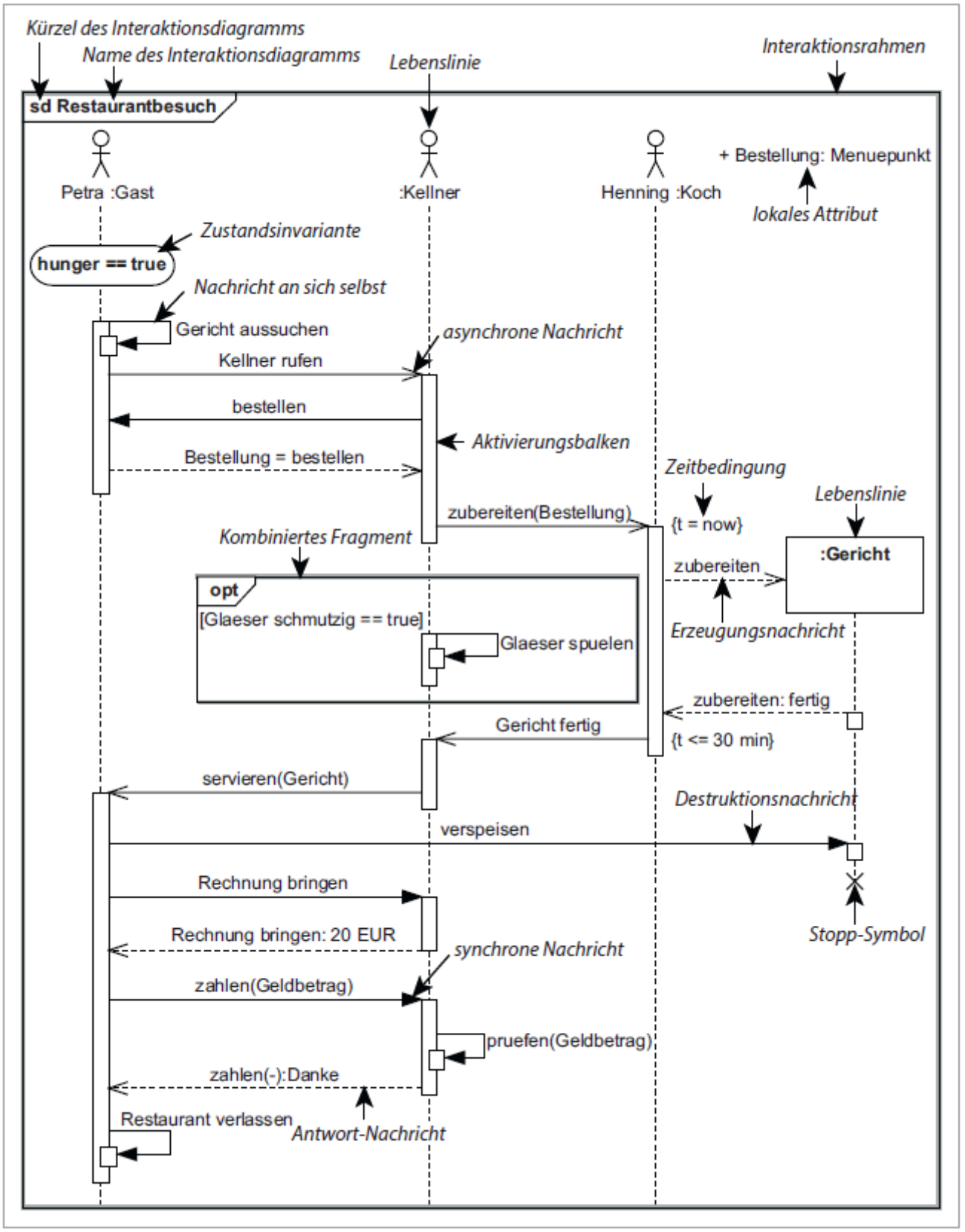
Sequenzdiagramme

Aus dem UML-Handbuch:

In der **Analyse** können Sequenzdiagramme verwendet werden, um Nachrichtenflüsse während der zu analysierenden Geschäftsprozesse abzubilden. Sie helfen damit dem Softwarearchitekten, zu verstehen, welche Personen und Systeme auf welche Weise miteinander interagieren, um bestimmte Aufgaben zu erfüllen. ...

Während der **Entwurfsphase** können Sequenzdiagramme gewinnbringend bei der Modellierung der Interaktion einzelner Systembestandteile eingesetzt werden. Häufig wird mit Sequenzdiagrammen auch die Interaktion von Benutzern mit der Benutzungsoberfläche der Software modelliert,...

Sequence Diagramm: Notationselemente



Sequence Diagramm: Identifiziere die Fehler!

